NODEBASE - INTERVIEW PREPARATION

Project Introduction

What Nodebase is, why it exists, who it is for, and what I learned while building it.

12 questions, answered in simple language

Each answer is written to be spoken in an interview. It is based on the actual codebase: Next.js 15, React 19, TypeScript, tRPC, Prisma, PostgreSQL, Better Auth, Polar, Inngest, BullMQ, Redis and React Flow.

Prepared in 2026

1. Tell me about your Nodebase project.

Nodebase is an AI workflow automation platform that I built to let people create automations visually, in a way that feels similar to n8n or Zapier. A user opens a canvas, drags a trigger node and one or more action nodes, connects them with edges, configures each node, saves the workflow, and then runs it. The trigger decides when a workflow starts, and the action nodes decide what should happen next.

The platform supports several triggers, including a manual button, a schedule, a Google Form submission, a Stripe event and a Telegram message. It supports actions such as HTTP requests, Google Sheets, OpenAI, Gemini, Anthropic, Discord, Slack and Telegram messages. When a workflow runs, the server loads the graph, sorts the nodes in the correct order, and passes a shared context object from one node to the next. Every node reads from that context and writes its result back into it.

I built the project end to end using Next.js 15, React 19, TypeScript, tRPC, Prisma and PostgreSQL, with Better Auth for authentication, Polar for subscriptions, Inngest for durable execution, BullMQ and Redis for background jobs, and React Flow for the editor. The main goal was to make automation understandable for people who do not want to write code or manage servers.

2. What problem does Nodebase solve?

Small teams often repeat the same work across several tools. A form submission has to be copied into a spreadsheet, summarised by an AI model, and then sent to a chat application. Doing that manually is slow, easy to forget, and hard to audit. Nodebase solves that by letting a user describe the sequence once on a visual canvas and then run it automatically whenever the trigger fires.

The project also solves the problem of visibility. When an automation fails, people usually do not know which step failed or why. Nodebase stores every run as an execution row with a status, an error message, a stack trace and the final output. The editor also shows the status of each node in real time while the workflow is running, so the user can see whether a node is loading, successful, or failed.

Finally, Nodebase solves the problem of connecting AI to everyday business tools. The AI nodes are not isolated demos. They sit in the same pipeline as HTTP, Sheets, Telegram and Slack nodes, so the output of an AI call can be sent somewhere useful immediately. That makes the platform practical rather than just a technical experiment.

3. Why did you decide to build Nodebase?

I wanted a project that would force me to learn how a real production application works from the database all the way to the browser. A simple CRUD application would not have been enough, because I wanted to understand background jobs, retries, real-time updates, authentication, payments and third-party integrations. Nodebase gave me a reason to use all of those pieces together in one system.

I was also personally interested in workflow tools. I had used automation platforms before, and I liked the idea of representing a process as a graph of nodes and edges. That model is easy to explain to a user, but it is technically interesting because the graph has to be validated, saved atomically, sorted, executed in order, and observed while it runs.

Another reason was portfolio value. I wanted to be able to explain not only what the product does, but also why each technology was chosen and what trade-offs I made. Building Nodebase gave me concrete stories about debugging email delivery, handling duplicate webhooks, encrypting credentials, recovering from failed jobs, and designing a database schema for a graph-shaped domain.

4. What are the main features of Nodebase?

The first major feature is the visual workflow editor. Users can add trigger and action nodes, connect them, move them around, configure them through dialogs, and save the entire graph. The editor also tracks unsaved changes and shows a live status badge on each node while a workflow is executing.

The second feature is the workflow engine. It creates an execution record, loads the workflow, topologically sorts the nodes, runs each node through an executor, threads a shared context through the graph, and saves the final output. It retries in production, records failures, and updates the execution history.

The third feature is integration support. There are triggers for manual runs, cron schedules, Google Forms, Stripe and Telegram, and actions for HTTP, Google Sheets, OpenAI, Gemini, Anthropic, Discord, Slack and Telegram. Credentials for those services are encrypted before they are stored.

The remaining features support the product around the editor: email and password authentication with OTP verification, Google and GitHub sign-in, Polar subscription gating, workflow and credential management, execution history, rate limiting, idempotency, and a queue dashboard for operational visibility.

5. Who are the target users of your project?

The main target users are small businesses and operations teams that depend on several online tools but do not have an engineering team. For example, a marketing team might want to take a form submission, summarise it with AI, and post it into Slack. A solo founder might want to receive a Telegram alert whenever a Stripe payment happens. These users need automation, but they do not want to build and host their own integration service.

A second group is developers and technical founders who want to prototype an integration quickly. The visual editor is faster than writing a small service for every experiment, and the execution history makes it easy to see what happened. Developers can also use the HTTP node to call their own APIs, which makes the platform extensible without needing a new node type for every service.

A third group is creators, educators and consultants who teach automation or build workflows for clients. They can reuse credentials, demonstrate a workflow visually, and show a clear run history. The product is not aimed at very large enterprises yet, because it does not have team roles, audit logs, or fine-grained permissions, but the core workflow model is designed in a way that could support those features later.

6. Explain the complete architecture of Nodebase.

Nodebase uses a full-stack Next.js application as the main entry point. The browser loads React Server Components for pages such as the workflow list, the editor, credentials and executions. Server components call the authentication guard, prefetch data through tRPC, and dehydrate that data into the React Query cache, so the client can render with `useSuspenseQuery` without an extra waterfall request.

The client talks to the server through tRPC. Inside the server, every request passes through a procedure chain. The base procedure is public. The protected procedure resolves the Better Auth session and rejects unauthenticated users. The premium procedure additionally checks Polar for an active subscription. Feature routers then call Prisma, which talks to PostgreSQL.

Workflow execution is separated from normal request handling. A trigger sends an event to Inngest. The Inngest function creates an execution row, loads the graph from PostgreSQL, sorts the nodes, and calls an executor for each node. Executors can call external services such as OpenAI, HTTP APIs, Telegram or Google Sheets, and they publish status updates through Inngest realtime channels. The browser subscribes to those channels using a short-lived token.

Supporting services complete the architecture. Better Auth stores users, accounts, sessions and verification rows in PostgreSQL. BullMQ and Redis run the OTP email worker and the cron worker. Polar handles subscriptions. Upstash Redis provides rate limiting and idempotency keys. The whole system is designed so that the interactive request path stays fast, while slower work happens in workers and Inngest.

7. Walk me through the flow of a user request in your application.

A good example is the manual Execute workflow button. The button is only shown when the canvas contains a manual trigger. When the user clicks it, the client calls the protected tRPC procedure `workflows.execute` with the workflow id. The procedure checks that the workflow exists and belongs to the signed-in user, then sends a `workflows/execute.workflow` event to Inngest.

The HTTP response returns quickly because the work has only been queued. Inngest then runs the `executeWorkflow` function. Its first step creates an Execution row with status RUNNING and a unique Inngest event id. The next step loads the workflow with its nodes and connections and runs topological sort. A cycle fails before any node executes. A separate step resolves the workflow owner because executors need the owner id to find the right encrypted credentials.

The function then starts with an empty context object, or with `initialData` if the run came from a webhook or cron. For each node in sorted order, it looks up the executor for that node type and calls it with the node data, node id, user id, current context, Inngest step tools and a publish function. The executor publishes a loading status, performs its work, merges its result into the context, and publishes success or error.

When all nodes finish, the function marks the execution SUCCESS, sets `completedAt`, and stores the final context in the output column. If a node throws and retries are exhausted, the `onFailure` hook marks the execution FAILED and saves the error and stack trace. In the browser, each node card listens to its realtime channel and updates its badge. The executions page reads the stored rows through tRPC.

8. What is the most interesting feature you implemented?

The most interesting feature for me is the execution engine. It looks simple from the outside, but it combines graph sorting, durable steps, retries, shared state, credential resolution and real-time feedback. The core idea is that the workflow context is a single JavaScript object. Each executor receives the current context and returns a new context that keeps the old keys and adds its own result under the variable name that the user configured.

That design is powerful because it lets one node use the output of any earlier node without a special connection format. For example, an HTTP node can store its result as `myApiCall`, a Gemini node can read `{{myApiCall.httpResponse.data}}`, and a Telegram node can read `{{myGemini.text}}`. Handlebars compiles those templates at execution time, when the real data is available, not when the workflow is saved.

The other interesting part is the realtime status channel. It is not required for correctness, but it changes the user experience completely. Instead of waiting for a run to finish and then refreshing a table, the user sees each node light up as it starts, succeeds, or fails. That required a separate realtime channel for every node type, a typed topic, a short-lived subscription token, and a shared hook that filters messages by node id. The combination of durable execution and live feedback is what makes the product feel like a real automation tool.

9. What was your role in developing the project?

I was the full-stack developer for the project. I designed the database schema, built the tRPC procedures, implemented the authentication flow, created the visual editor, wrote the node executors, connected the background workers, and prepared the deployment configuration. I also worked on the less visible production concerns, such as encryption, idempotency, rate limiting, error handling, logging and subscription checks.

On the frontend, I built the React Flow canvas, the node selector, the configuration dialogs, the workflow list, the credentials pages and the executions pages. I used React Query hooks and server prefetching so the pages load with data already available. I also handled the editor's dirty state, so the Save button only enables when the graph has actually changed.

On the backend, I implemented the procedure chain, ownership checks, Prisma transactions for saving a workflow, credential encryption with Cryptr, and the Inngest execution function. I also set up the BullMQ workers for email OTP delivery and cron scheduling, and the webhook routes for Google Forms, Stripe and Telegram.

Finally, I was responsible for debugging the system when it did not work. That included inspecting live database rows, reading Redis queue state, tracing failed SMTP deliveries, and checking Inngest retry behaviour. In a real team I would be comfortable owning a feature from the database schema to the UI.

10. What was the most challenging part of the project?

The most challenging part was making the whole execution path reliable, not just writing individual nodes. In a demo, it is enough to call an API and show the result. In a real workflow product, the same run may be retried, a webhook may arrive twice, a node may fail halfway through, and the user still needs to see a correct execution record. That forced me to think about idempotency, durable steps, error classification and the difference between a retryable failure and a permanent configuration error.

The second challenge was the authentication and email flow. Signing up looked successful, but no session was created until the email was verified. The OTP email was sent by a separate worker process, not by the web request. A misconfigured trusted origin returned the same 403 status as an unverified email, so the client silently redirected to the verification page. Later, the SMTP provider rejected the credentials. Debugging that required checking each hop: the auth route, the verification row, the BullMQ queue, the failed job reason and the SMTP configuration.

The third challenge was keeping the editor and the database in agreement. The editor has three sources of state: React state, React Flow's internal store, and the save button's snapshot. I had to understand how React Flow propagates changes, how to compute the dirty flag, and how to save the graph without wiping node configuration. That taught me that the hardest part of a workflow editor is not drawing nodes, but preserving data correctly across saves.

11. What did you learn while building Nodebase?

I learned how much of production engineering happens outside the happy path. Before this project, I thought about features mainly as screens and endpoints. Now I think about retries, duplicate events, partial writes, expired credentials, rate limits, background workers, and what the user sees when something fails. Those concerns shaped the database schema and the execution engine as much as the product requirements did.

I learned how to use Next.js App Router properly, including server components, route groups, server prefetching and hydration. I also learned how tRPC keeps types consistent from the database to the client, and how Prisma transactions, cascading deletes and unique constraints protect data integrity. Working with Inngest taught me the value of step memoization: a retry should not repeat work that already succeeded.

I learned a lot about security and operations as well. Credentials are encrypted at rest and decrypted only inside executors. Webhooks use rate limiting and idempotency keys. Queue workers need retry policies, graceful shutdown and failed-job inspection. I also learned to debug by reading the actual state of the system rather than guessing, whether that meant querying PostgreSQL metadata, inspecting Redis keys, or reading the installed library source. That habit is probably the most valuable thing I took from the project.

12. If you had more time, what would you improve?

The first improvement would be automated testing. The project currently relies on linting, type checking, production builds and manual end-to-end checks. I would add unit tests for the executors and validation schemas, integration tests against a temporary PostgreSQL database, and Playwright tests for sign-up, workflow creation, saving and execution. I would also test the failure paths, because those are the paths that are hardest to verify by hand.

The second improvement would be reliability and correctness. I would route manual executions through the shared dispatch helper so they get an idempotency key, set `completedAt` when a run fails, validate cycles at save time instead of only at execution time, and fix the node selector branch that can wipe a canvas when a legacy INITIAL node exists. I would also fix the OpenAI executor publishing to the Anthropic channel.

The third improvement would be architecture and operations. I would replace the near-duplicate realtime channel and token files with a generic factory, scope realtime subscriptions to a run instead of one subscription per node, add a credential key-rotation path, and encrypt OAuth tokens. I would add structured logging, Sentry alerts, queue depth monitoring, backups and a proper CI/CD pipeline. Finally, I would add team workspaces and role-based access so the product can serve larger customers.